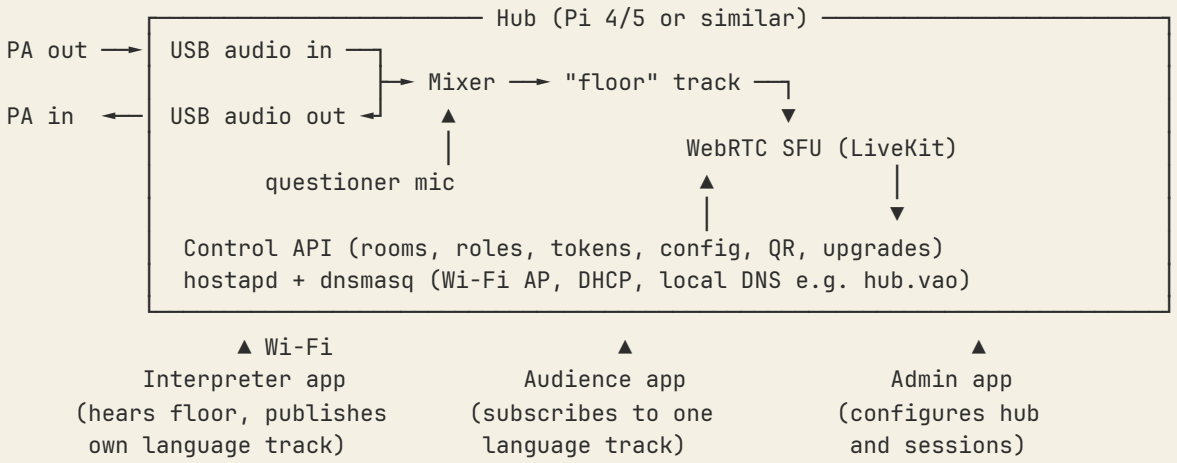
# Boca a Oreja — Development Plan

A self-contained, low-cost system for simultaneous interpretation using smartphones. See `System Overview Description.odt` for the product concept and roles.

## Goals

- A **room hub** (Raspberry Pi or any small computer with Wi-Fi) that acts as the Wi-Fi access point and routes all audio between phones.
- **Android and iOS apps** for administrators, interpreters, audience and questioners.
- Works fully offline; Internet is only used for hub upgrades.
- Low enough latency for live interpretation.

## Architecture




### Hub components

| Component | Role | Candidate tech |
|---|---|---|
| Access point | Wi-Fi network, DHCP, local DNS, client isolation | hostapd, dnsmasq |
| Media server | Receives each stream once, forwards to all subscribers | LiveKit (single Go binary, ARM64) |
| Mixer | Floor = PA input + active questioner; questioner also to PA output | GStreamer or PipeWire |
| Control API | Accounts, roles, session config, JWT tokens, QR codes, upgrades | Go (or Python for prototyping) |
| Web client | Test page early on; possibly the audience client later | Served by the control API |
| Upgrade agent | Always downloads upgrades; installs automatically or on admin action | RAUC / Mender / signed apt repo |

### Audio channels

- **Floor**: PA (or network) input mixed with the active questioner → all interpreters.
- **Language N**: active interpreter for language N → audience members who chose N.
- **Questioner**: audience member's mic → mixer → floor track and PA output.

### Why WebRTC instead of RTSP/RTMP

The early notes suggested RTSP. WebRTC is the better fit:

- **Latency**: roughly 100–300 ms on a LAN, versus about a second or more for typical RTMP/RTSP pipelines. Interpreters need the floor audio as fast as possible.
- **Audio quality**: Opus, echo cancellation, gain control and jitter buffering built in.
- **Platforms**: mature SDKs for iOS, Android and browsers (the RTSP library in `Notes` is Android-only).

An SFU (Selective Forwarding Unit) means each phone sends one stream and the hub forwards it to everyone subscribed. LiveKit's JWT tokens map directly onto the roles.

### Routing

Phones only talk to the hub; client isolation is enabled on the AP and the SFU does all routing. Client traffic is not NATed to the Internet. An uplink (Ethernet or second Wi-Fi interface) is used only by the hub itself, for upgrades.

## Phases

### Phase 0 — Feasibility spikes

Answer the questions that can sink the project before writing much code.

1. **Wi-Fi client capacity.** The Pi's built-in Wi-Fi supports only a low number of clients in AP mode (often quoted as roughly 10–20). Measure:
   - built-in Wi-Fi,
   - a USB adapter with good AP support (mt76-based adapters are commonly recommended),
   - a dedicated access point with the Pi as server only.
2. **End-to-end latency.** PA → hub → interpreter → hub → listener, with LiveKit on a Pi.
3. **Phone behaviour on a Wi-Fi network without Internet.**
   - Android may move traffic to mobile data; check whether the app must bind to the Wi-Fi network explicitly.
   - iOS needs the Local Network permission.
   - Background playback with the screen locked on both platforms.
4. **Hub load.** CPU, memory and bandwidth with the target number of listeners.

**Exit criteria:** known maximum audience per hub for each Wi-Fi option, measured latency within target, no blocking OS restrictions found.





## Phase 1 — Hub MVP

- OS image based on Debian / Raspberry Pi OS Lite.
- hostapd + dnsmasq configured as AP with local DNS name.
- LiveKit running as a systemd service.
- One USB audio input published as the floor track.
- One hardcoded language channel.
- Web test page to listen and talk, before any app exists.

**Exit criteria:** an interpreter and listeners can use the system end to end from a browser on the hub's network.

## Phase 2 — App MVP (listener + interpreter)

- Flutter app using `livekit_client`, one codebase for Android and iOS.
- **Listener**: discover hub, choose language, play audio, keep playing in background.
- **Interpreter**: hear floor in headset, mic on/off, handover between interpreters sharing a language (normally only one is live at a time).
- Join via QR code containing hub address and session.

**Exit criteria:** a session with several interpreters and listeners runs on real Android and iOS phones.

## Phase 3 — Questioner flow

- Audience member requests to speak.
- Moderator/admin approves; the questioner's mic is routed into the mixer, to the PA output and the floor track.
- Automatic mute when the question ends or is revoked.

**Exit criteria:** a question is heard in the room and interpreted into all languages.

## Phase 4 — Administration and configuration

- First-boot setup from the default configuration, creating admin accounts.
- Per-session configuration: SSID, password, room/session name, language channels, main audio input.
- Event files with all sessions/rooms; every hub at an event holds the full event. Import/export first, hub-to-hub sync later.
- QR code generation for audience, interpreters and administrators.

**Exit criteria:** a multi-room event can be prepared in advance and loaded onto all hubs.

## Phase 5 — Packaging and upgrades

- Flashable SD-card image built with pi-gen.
- Upgrades are always downloaded when online; installed automatically or by an admin, depending on configuration.
- A/B partitions with rollback (RAUC or Mender), or a signed apt repository as a simpler start.

**Exit criteria:** a fresh image can be flashed, set up and upgraded without a terminal.





### Phase 6 — Pilot and release

- Pilot at a real event; collect latency, dropout and usability feedback.
- Play Store and App Store releases. Allow time for Apple review, and provide a demo mode since the app needs hardware the reviewer won't have.

## Open decisions

1. **Audience size per room** — decides the Wi-Fi hardware (built-in, USB adapter, external AP).
2. **Audience without app install** — listeners could use a web page served by the hub. Receive-only WebRTC works over plain HTTP; interpreters and questioners still need the app, since browsers only allow microphone access over HTTPS.
3. **Flutter or native Swift/Kotlin** — Flutter recommended unless its audio handling falls short.
4. **Hub service language** — Go fits alongside LiveKit; Python is faster to prototype.

## Risks

| Risk | Mitigation |
|---|---|
| Built-in Wi-Fi can't serve a full room | USB adapter or external AP; measure in Phase 0 |
| Latency too high for interpretation | Measure early; tune Opus frame size, jitter buffer |
| Phones drop or reroute an offline Wi-Fi network | Bind to network in app; test across OS versions |
| Background audio killed by the OS | Android foreground service; iOS background audio |
| App store rejection | Demo mode, clear review notes |
| Interpreter hears own voice / echo | Headsets required; echo cancellation on |